PRACTICAL 9

```
#include <stdio.h>
#include <stdbool.h> // for bool type

// Function to find the page that will not be used for the longest period of time
int findOptimal(int pages[], int memory[], int n, int current_index, int frame_count)
{
    int res = -1, farthest = current_index;

    for (int i = 0; i < frame_count; i++)
    {
        int j;
        for (j = current_index; j < n; j++)
        {
            if (memory[i] == pages[j])
            {
                if (j > farthest)
                {
                    farthest = j;
                    res = i;
                }
                break;
            }
        }

        // If a page is never referenced again, it is the best candidate for replacement
        if (j == n)
            return i;
    }

    // If all pages will be used again, return the one with the farthest next use
    return (res == -1) ? 0 : res;
}

int main()
{
    // Page reference string
    int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    int n = sizeof(pages) / sizeof(pages[0]);
    int frame_count = 3; // As specified (No of Frames = 3)
    int memory[3];
    int hits = 0, faults = 0, occupied = 0;

    // Initialize memory slots with -1 (empty)
    for (int i = 0; i < frame_count; i++)
```

```
    memory[i] = -1;

printf("Optimal Page Replacement Simulation\n");
printf("-----------------------------------\n");
printf("Ref | Memory Status | Result\n");
printf("-----------------------------------\n");

for (int i = 0; i < n; i++)
{
    bool isHit = false;

    // Check if page is already in memory (Hit)
    for (int j = 0; j < occupied; j++)
    {
        if (memory[j] == pages[i])
        {
            isHit = true;
            hits++;
            break;
        }
    }

    // If not in memory (Fault/Miss)
    if (!isHit)
    {
        if (occupied < frame_count)
        {
            // Fill empty slots first
            memory[occupied++] = pages[i];
        }
        else
        {
            // Apply Optimal replacement logic
            int index = findOptimal(pages, memory, n, i + 1, frame_count);
            memory[index] = pages[i];
        }
        faults++;
    }

    // Display current state
    printf("%2d  | ", pages[i]);
    for (int j = 0; j < frame_count; j++)
    {
        if (memory[j] != -1)
            printf("%d ", memory[j]);
        else
            printf("- ");
    }
```

```
        printf(" | %s\n", isHit ? "HIT" : "FAULT");
    }

    // Calculate Percentages
    float hit_percent = ((float)hits / n) * 100;
    float fault_percent = ((float)faults / n) * 100;

    printf("-----------------------------------\n");
    printf("Total Page Hits  : %d (%.2f%%)\n", hits, hit_percent);
    printf("Total Page Faults: %d (%.2f%%)\n", faults, fault_percent);

    return 0;
}
```

OUTPUT

Output

```
Optimal Page Replacement Simulation
-----------------------------------
Ref | Memory Status | Result
-----------------------------------
 1  | 1 - -  | FAULT
 2  | 1 2 -  | FAULT
 3  | 1 2 3  | FAULT
 4  | 1 2 4  | FAULT
 1  | 1 2 4  | HIT
 2  | 1 2 4  | HIT
 5  | 1 2 5  | FAULT
 1  | 1 2 5  | HIT
 2  | 1 2 5  | HIT
 3  | 3 2 5  | FAULT
 4  | 4 2 5  | FAULT
 5  | 4 2 5  | HIT
-----------------------------------
Total Page Hits  : 5 (41.67%)
Total Page Faults: 7 (58.33%)


=== Code Execution Successful ===
```